A Sampling-and-Filtering Framework with Offline Policy Exploration for Certified Loop Invariant Generation

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

March 13, 2026

Abstract

Loop invariant generation is a central challenge in program verification. Existing LLM-based approaches rely on extensive SMT solving, complex workflows, and the most powerful closed-source models, yet still struggle with this task—which demands logical reasoning, program comprehension, and mastery of formal annotation syntax. We present SAM2INV, a system that trains a small, locally deployable model to generate high-quality numerical loop invariants through offline knowledge distillation with verifier feedback. To overcome training-data scarcity, we design LOOP-FACTORY, a probabilistic DSL constrained by semantic templates that synthesises structured numeric loop programs at arbitrary scale. We then build a multi-stage sampling-and-filtering pipeline: N parallel LLM calls are decomposed into conjuncts, unioned into a single pool, and filtered through syntax validation, execution-trace semantic checking, semantic deduplication, Houdini formal pruning via Frama-C/WP, and trivial-invariant exclusion—producing certified annotations. Rather than applying reinforcement learning—whose binary verifier reward is too sparse, reward-hackable, and ineffective for weak base models—we adopt a distillation strategy: a lightweight teacher model explores multiple candidates per program during data preparation; the verification pipeline filters them into chosen and rejected examples with reverse chain-of-thought synthesis; and the student is fine-tuned via SFT followed by DPO, with on-policy negative-sample augmentation, transferring the teacher's exploration benefit with no human annotation or online reward model.

1 Introduction

Loop invariant generation is a central challenge in program verification. Traditional approaches [Cousot and Cousot, 1977, Colón et al., 2003, Flanagan and Leino, 2001, Ernst et al., 2007, Nguyen et al., 2012] handle specific loop families but generalise poorly and incur prohibitive analysis time. Recent LLM-based methods [Chakraborty et al., 2024, Pei et al., 2023, Kamath et al., 2023] bring fresh perspectives, yet universally rely on extensive SMT solving, complex multi-round workflows, and the most powerful closed-source models—revealing that the task remains exceptionally hard even for LLMs, demanding *logical reasoning*, *program comprehension*, and *formal annotation mastery* simultaneously.

The heavy dependence on closed-source APIs creates deployment bottlenecks: a single project may contain hundreds of loops, each requiring multiple sampling-and-verification rounds, incurring prohibitive token costs and latency. API uncontrollability—model updates, service interruptions, data privacy—further limits safety-critical deployment. Training a *locally deployable* small model is therefore of considerable practical value.

We focus on *numerical* loop invariants—linear or polynomial inequalities over program variables whose verification is SMT-automatable. This preserves sufficient reasoning difficulty while keeping the formalisation barrier low, offering a unique lens to examine small models' integrated mathematical and programming abilities.

Realising this goal faces four challenges:

1. **Data scarcity.** Existing annotated corpora are extremely limited and carry data-leakage risks.
2. **Sparse feedback.** The SMT verifier yields a binary pass/fail verdict; hand-crafted reward shaping introduces human-prior bias and alignment risk.
3. **Reward hackability.** Trivially weak annotations (e.g. `loop invariant \true;`) pass the check yet capture no program property.
4. **Base-model bottleneck.** GRPO-style RL [DeepSeek-AI, 2025] requires multiple correct rollouts, but weak models rarely produce any within a practical sampling budget.

These constraints favour *knowledge distillation* over RL [DeepSeek-AI, 2025]: distillation transfers the teacher's reasoning trajectories directly, bypassing the autonomous exploration bottleneck. We choose GPT-5-nano as the teacher—bridging the student–teacher gap [DeepSeek-AI, 2025] while keeping per-program token cost tractable.

To address data scarcity, we build LOOPFACTORY, a DSL-based generator that synthesises diverse numerical loop programs with known semantic structure, eliminating data leakage by construction (Section 5). We then construct a unified *sampling-and-filtering pipeline* (Section 3) that serves both training data preparation and test-time inference, differing only in the scale of parallel teacher sampling. The pipeline decomposes candidates into conjuncts, filters them through syntax, trace-semantic, deduplication, Houdini, and triviality stages, and synthesises reverse chains-of-thought for the certified output. Survivors form *chosen* samples; rejects become *rejected* samples for DPO. Two augmentation strategies—off-policy conjunct recombination and on-policy student negatives—further enrich the preference signal. A two-stage SFT + DPO recipe completes the distillation (Section 4).

Contributions.

1. **LOOPFACTORY**: a probabilistic DSL for numerical loop synthesis overcoming data scarcity and leakage (Section 5).
2. **A multi-stage sampling-and-filtering pipeline** integrating verifier feedback, semantic deduplication, trivial invariant filtering, and reverse CoT synthesis to produce certified annotations (Section 4).
3. **An offline distillation strategy** with on-policy augmentation, producing SFT + DPO training data grounded in formal verification without online RL (Section 4.5).

2 Related Work

Classical Invariant Generation. Abstract interpretation [Cousot and Cousot, 1977] computes fixed-point over-approximations in abstract domains but is restricted to the chosen domain's expressiveness. Template-based constraint solving [Colón et al., 2003, Sankaranarayanan et al., 2004] searches for coefficients via LP/SDP, complete within the template family but blind outside it. The Houdini algorithm [Flanagan and Leino, 2001] iteratively removes non-inductive conjuncts from a candidate set; we adopt it as Stage F_4 of our filter pipeline. Daikon [Ernst et al., 2007] and DIG [Nguyen et al., 2012] mine likely invariants from traces but require a separate verifier pass. ICE learning [Garg et al., 2016] uses implication counterexamples to guide decision-tree learners, achieving completeness within its feature space. All these methods are confined to their pre-designed search spaces.

LLM-Based Invariant Generation. Chakraborty and Lahiri [Chakraborty et al., 2024] show GPT-4 can generate invariants with a verification feedback loop. Pei et al. [Pei et al., 2023] find iterative refinement essential for Hoare-style annotations. Kamath et al. [Kamath et al., 2023] combine LLM generation with bounded model checking. Endres et al. [Endres et al., 2024] study whether LLMs can write formal specifications in general, finding success on simpler templates but frequent failures on complex loop invariants. LLMs also suggest proof tactics [Han et al., 2022, Jiang et al., 2023] and produce formal proofs [Wu et al., 2022, Lample et al., 2022, Polu et al., 2023]. A common thread is heavy reliance on external tool chains and the strongest closed-source models, without exploring locally deployed small models.

Learning-Based Loop Invariant Synthesis. Si et al. [Si et al., 2018] frame invariant generation as a supervised learning task, training a graph neural network on programs from the SyGuS competition and using counterexample-guided refinement. Code2Inv [Si et al., 2020] extends this with a continuous representation of programs and invariants, using reinforcement learning with the Z3 verifier as the reward signal. CLN2INV [Ryan et al., 2020] learns continuous logic networks that map program variables to invariant satisfaction scores, achieving competitive performance on polynomial invariants. These approaches pioneered neural invariant synthesis but are constrained to fixed template families (e.g., octagonal or polynomial) and do not leverage the flexible generation capabilities of pretrained language models.

Mathematical Reasoning Distillation. Knowledge distillation for mathematical reasoning has attracted significant attention. The DeepSeek-R1 distilled models [DeepSeek-AI, 2025] demonstrate that distillation transfers reasoning trajectories to small models more effectively than RL, which requires the student to rediscover them through sparse rewards. Mukherjee et al. [Mukherjee et al., 2023] show that step-by-step explanations from GPT-4 substantially improve small model reasoning via imitation learning. WizardMath [Luo et al., 2023] applies RLHF with process supervision to enhance mathematical reasoning in open models. Yuan et al. [Yuan et al., 2023] demonstrate that rejection sampling from large models, combined with SFT on verified solutions, offers a scalable alternative to RL for math. MAmmoTH [Yue et al., 2024] constructs instruction-tuning data by combining CoT and program-of-thought rationales from GPT-4, achieving strong math reasoning in 7B–70B models. Our approach inherits the RLVR spirit but moves exploration entirely offline, using formal verifier feedback

SAM2INV Pipeline

❶ **Dynamic Sampling** — Execute with tiered inputs; collect per-iteration variable traces.

❷ **Parallel LLM Generation** — N calls with diverse temperatures; decompose into conjuncts and union.

❸ **ACSL Syntax Filter** — Reject structurally invalid conjuncts.

❹ **Trace Semantic Filter** — Discard conjuncts falsified by observed states.

❺ **Semantic Deduplication** — Collapse logically equivalent conjuncts via auxiliary LLM.

❻ **Houdini Formal Pruning** — Iteratively remove non-inductive conjuncts via Frama-C/WP.

❼ **Trivial Invariant Filter** — Exclude formally correct but semantically vacuous invariants.

❽ **Reverse CoT & Distillation** — Synthesise reasoning chains; construct chosen/rejected pairs; augment and train via SFT + DPO.

Figure 1: SAM2INV pipeline. ❶–❷: generation; ❸–❼: multi-stage filtering; ❽: distillation.

to construct ground-truth preference pairs for SFT + DPO distillation specifically for the program verification domain.

Distillation vs. RL for Code and Reasoning. RLHF [Ouyang et al., 2022] and DPO [Rafailov et al., 2023] are standard LLM alignment techniques. RLVR replaces human preferences with automated oracle rewards, yielding large gains in math reasoning (DeepSeek-R1 [DeepSeek-AI, 2025], Tulu 3 [Lambert et al., 2024]). CodeRL [Le et al., 2022] and RLEF [Gehring et al., 2024] apply RL with execution feedback for code generation. However, binary verifier rewards are particularly sparse for loop invariant generation: a single missing conjunct invalidates the entire annotation. This sparsity, combined with the reward-hackability of trivially weak invariants, motivates our offline distillation approach over online RL.

Training Data Synthesis. Existing benchmarks (NLA suite [Beyer, 2023], linear set [Si et al., 2018]) contain only dozens of hand-crafted programs. Grammar-based fuzzing [Godefroid et al., 2008] and probabilistic synthesis [Nori et al., 2015] generate programs at scale but lack semantic constraints. LOOPFACTORY addresses this gap with semantic core templates that guarantee well-defined invariant structure at tunable complexity.

3 System Overview

SAM2INV is built around a single unified pipeline (Figure 1) that serves *both* training-data synthesis and test-time inference. Given a C function with a `while` loop, precondition, and postcondition, the pipeline outputs ACSL-annotated code verified by Frama-C/WP. During training data preparation it additionally produces SFT and DPO records; at inference time those stages are simply deactivated. Table 1 summarises which components are active in each mode.

Phase I: Generation (❶–❷). The program is first executed under tiered inputs to collect execution traces $\mathcal{T}$. The value domain is partitioned into four tiers of increasing magnitude—corner values $(0, \pm 1)$, small positives (2–5), medium values (6–50), and large values (51–

Table 1: Component activation under the two operating modes. The generation–filtering pipeline (❶–❼) is shared; distillation components (❽) are training-only.

Component	**Training**	**Inference**
Dynamic Sampling (❶)	✓	✓
LLM caller (❷)	teacher $\mathcal{M}_T$, N large	student $\mathcal{M}_S$, N small
Multi-stage filtering (❸–❼)	✓	✓
Training-only (❽):		
Reverse CoT synthesis	✓	—
DPO pair construction	✓	—
Data augmentation	✓	—

500)—and inputs are drawn in phases of increasing tier complexity to cover both boundary and interior behaviour. To fit LLM context limits, each trace retains the first and last k iterations (default k=3); traces are grouped into at most G groups (default G=10). A structured *loop context* is then assembled from the traces, a static-analysis record (loop guard, variable scopes, pre-condition), and a symbolically derived transition relation (Section 4.1). N parallel LLM calls at diverse temperatures each return a CoT reasoning trace and candidate invariants; all invariant expressions are decomposed into conjuncts and unioned into a single pool C_{pool}. During training, the teacher $\mathcal{M}_T$ is called with a large N to maximise conjunct coverage; at inference, the student $\mathcal{M}_S$ is called with a smaller N.

Phase II: Multi-Stage Filtering (❸–❼). Five filters of increasing cost refine the pool to a certified, non-trivial invariant set C^* (Section 4.3). Every surviving conjunct is formally verified by Frama-C/WP. This phase is identical in both modes.

Phase III: Distillation (❽). Active only during training. A reverse CoT model synthesises a reasoning chain for C^*. The result forms the *chosen* sample; rejected candidates become DPO negatives (Section 4.5). Tiered sampling details are in Appendix F.

4 Method

Notation. A program $P = (\phi_{\text{pre}}, G, B, \phi_{\text{post}})$ has variables V, precondition ϕ_{pre}, guard G, body B, and postcondition ϕ_{post}. An execution trace $\tau = (\sigma_0, \ldots, \sigma_T)$ records per-iteration states $\sigma_t : V \to \mathbb{Z}$. A candidate invariant $I = c_1 \wedge \cdots \wedge c_k$ is *inductive* (written $\text{Ind}(I, P)$) if $\phi_{\text{pre}} \Rightarrow I,\ I \wedge G \wedge B \Rightarrow I',$ and $I \wedge \neg G \Rightarrow \phi_{\text{post}}$.

4.1 Loop Context Construction

Before calling the LLM, we assemble a structured *loop context* that provides all information needed for invariant synthesis. A lightweight static pre-pass (`LoopAnalysis`) extracts a per-loop *record* $\mathbf{r}_i$ containing: (i) the loop guard and body code snippet, (ii) the set of modified variables V_{mod} and unmodified function parameters $\mathcal{P}$, (iii) conservative integer bounds on each variable, and (iv) the loop entry pre-condition ϕ_{pre}.

A symbolic execution pass then enriches each record with a *transition relation*: for each variable the pass computes the one-step mapping $v \mapsto e'(v, \mathcal{P})$ under the loop guard,

distinguishing changed variables from unchanged ones. This gives the LLM a concise algebraic view of the loop body beyond raw source code.

The final prompt concatenates four components: (i) the ACSL pre-condition ϕ_{pre}, (ii) the transition relation summary, (iii) the loop code snippet, and (iv) execution traces with temporal labels (BEFORE iteration / AFTER termination) emphasising that invariants must hold at the *start* of every iteration. Available variables and function parameters (supporting `\at(var, Pre)` references) are listed explicitly so that the LLM only uses well-scoped identifiers. The full prompt template is given in Appendix D.

4.2 Parallel LLM Generation

We issue N parallel calls to the teacher $\mathcal{M}_T$ with temperatures drawn from $\{0.8, 1.0, 1.1, 1.2\}$. Each call j returns $r_j = (\rho_j, I_j)$: a CoT reasoning trace ρ_j and a candidate invariant I_j. Candidates are decomposed into conjuncts and unioned:

$$C_{\text{pool}} = \bigcup_{j=1}^{N} \text{conjuncts}(I_j) \;/\; \sim_{\text{ws}} \tag{1}$$

where $\sim_{\text{ws}}$ is whitespace-normalised string equality. The union exploits the conjunctive structure of invariants: distinct calls discover complementary conjuncts. Prompt design and temperature strategy are detailed in Appendix D.

4.3 Multi-Stage Filtering

The pool is refined through five stages of increasing cost:

$$C_{\text{pool}} \xrightarrow{F_1} C_1 \xrightarrow{F_2} C_2 \xrightarrow{F_3} C_3 \xrightarrow{F_4} C_4 \xrightarrow{F_5} C^*$$

Algorithm 1 presents the complete procedure.

F_1**: Syntax filter.** $C_1 = \{c \in C_{\text{pool}} \mid \text{SynValid}(c)\}$ checks ACSL structural rules (Appendix E). Cost: $O(|c|)$ per conjunct.

F_2**: Trace semantic filter.** $C_2 = \{c \in C_1 \mid \forall \tau \in \mathcal{T}, \forall \sigma \in \tau : \sigma \models c\}$. Sound (no true invariant removed) but incomplete.

F_3**: Semantic deduplication.** An auxiliary LLM identifies equivalence classes under logical equivalence $\equiv_{\text{sem}}$ and retains one representative per class: $C_3 = \{\text{repr}([c]_{\equiv_{\text{sem}}}) \mid c \in C_2\}$.

F_4**: Houdini pruning.** The Houdini algorithm [Flanagan and Leino, 2001] iteratively removes conjuncts whose verification conditions fail in Frama-C/WP until a fixed point C_4.

Theorem 1 (Termination)**.** *Houdini terminates in* $\leq |C_3|$ *iterations, removing* ≥ 1 *conjunct per round.*

F_5**: Trivial invariant filter.** A quality discriminator $\mathcal{M}_Q$ excludes invariants that are formally correct but semantically vacuous, preventing reward hacking.

Input: Program P, teacher $\mathcal{M}_T$, verifier $\mathcal{V}$, quality judge $\mathcal{M}_Q$
Output: Certified invariant C^* or $\perp$; training records

$\mathcal{T} \leftarrow \textsc{TieredSample}(P)$ ▷ *Phase I*
for $j = 1, \ldots, N$ **in parallel do**
  $(\rho_j, I_j) \leftarrow \mathcal{M}_T(P, \mathcal{T}, t_j)$
$C_{\text{pool}} \leftarrow \bigcup_j \text{conjuncts}(I_j) \ / \sim_{\text{ws}}$

$C_1 \leftarrow \{c \in C_{\text{pool}} \mid \text{SynValid}(c)\}$ ▷ F_1*: syntax*
$C_2 \leftarrow \{c \in C_1 \mid \forall \sigma \in \mathcal{T} : \sigma \models c\}$ ▷ F_2*: trace*
$C_3 \leftarrow \text{SemDedup}(C_2)$ ▷ F_3*: dedup*

$S \leftarrow C_3$ ▷ F_4*: Houdini*
repeat
  $R \leftarrow \{c \in S \mid \mathcal{V}(P, \bigwedge S, c) = \textit{fail}\}$
  $S \leftarrow S \setminus R$
until $R = \emptyset$ **or** $S = \emptyset$
$C_4 \leftarrow S$

$C^* \leftarrow C_4$ **if** $\mathcal{M}_Q(P, C_4) = \textsc{ok}$, else $\emptyset$ ▷ F_5*: trivial*

$\rho^* \leftarrow \mathcal{M}_R(P, C^*)$ ▷ *reverse CoT*
return $C^*,\ y^+ = (\rho^*, C^*),\ \{y^-\} = C_{\text{pool}} \setminus C^*$

Algorithm 1: SAM2INV Pipeline

4.4 Reverse Chain-of-Thought

The certified C^* is assembled from fragments of multiple teacher responses and differs from any single I_j. A reverse CoT model $\mathcal{M}_R$ synthesises a matching reasoning chain: $\rho^* = \mathcal{M}_R(P, C^*)$, ensuring every training sample is a complete (ρ, I) pair.

4.5 Offline Knowledge Distillation

For each program P the pipeline yields:

- **Chosen**: $y^+ = (\rho^*, C^*)$ — certified invariant with reverse CoT.
- **Rejected**: $\{y_s^-\}$ — every candidate eliminated at stage s.

Data augmentation. (i) *Off-policy conjunct recombination* (Algorithm 4): the pipeline collects a *negative pool* B of every conjunct rejected at any stage (syntax, trace, Houdini). Rejected variants are constructed by independently sampling a subset of correct conjuncts and a subset of incorrect ones, subject to the constraint that the result is not equivalent to the chosen annotation (Appendix C.2). (ii) *On-policy negatives*: the student $\mathcal{M}_S$ generates candidates through the same pipeline; failures serve as targeted rejected samples close to the student's current distribution.

Two-stage training. *SFT* minimises negative log-likelihood on chosen samples:

$$\mathcal{L}_{\text{SFT}} = -\sum_{(P, y^+)} \log \pi_\theta(y^+ \mid P). \tag{2}$$

Table 2: Semantic families of LOOPFACTORY templates. Each family encodes a distinct proof obligation pattern with guaranteed termination.

Family	**Invariant Pattern**	**Example**
Linear conservation	$a + b = N$	transfer loops
Monotone accumulation	$s \mathrel{+}= f(k)$	prefix sums
Proportional stride	$j = c \cdot i$	scaled counters
Modular counting	$c \equiv t \pmod{T}$	cyclic counters
State-machine transition	finite-state updates	flag toggles
Division / GCD	$a = q \cdot d + r$	Euclidean division
Multiplicative redistr.	$z + xy = a_0 b_0$	Russian multiply
Polynomial recurrence	$zw \geq p^2$	Cauchy–Schwarz

DPO [Rafailov et al., 2023] then refines preferences from the SFT checkpoint π_{ref}:

$$\mathcal{L}_{\mathrm{DPO}} = -\sum_{(P,y^+,y^-)} \log \sigma\Big(\beta\big[\log \frac{\pi_\theta(y^+ \mid P)}{\pi_{\mathrm{ref}}(y^+ \mid P)} - \log \frac{\pi_\theta(y^- \mid P)}{\pi_{\mathrm{ref}}(y^- \mid P)}\big]\Big). \tag{3}$$

The pipeline certifies every output regardless of model quality: a weaker model yields a smaller C^* but never an uncertified one. Stage-stratified DPO weighting and further details are in Appendix C.

5 Numerical Loop Synthesis

To overcome the scarcity of annotated numeric loop programs, we design LOOPFACTORY, a probabilistic DSL that synthesises well-formed C loop programs at arbitrary scale. A program has the form **Init**; **Loop**$^+$, where loops may be sequential or nested; structural complexity is freely tunable via fuel parameters governing the number of variables, loops, assignments, and conditional blocks per program. Because uniform sampling from the grammar overwhelmingly produces divergent or semantically vacuous programs, we constrain generation with *semantic core templates*: each template fixes a termination-guaranteed update skeleton and a known invariant shape, while leaving variable names and constants to probabilistic instantiation.

Table 2 summarises the eight proof-obligation families covered by the 60 templates, ensuring that the corpus spans the full range of arithmetic loop patterns an invariant-generation policy must handle.

Using LOOPFACTORY we synthesise 2,000 programs; 1,900 are used for training and 100 are held out as a synthetic test set. Full DSL definition, template descriptions, and generation examples are given in Appendix A.

6 Experiments

7 Conclusion

We presented SAM2INV, a system for certified numerical loop invariant generation comprising three contributions: LOOPFACTORY for scalable training data synthesis; a multi-stage sampling-and-filtering pipeline producing formally verified, non-trivial invariants; and an offline distillation strategy that embeds verifier feedback into SFT + DPO training data via

reverse CoT synthesis and on-policy augmentation—with no human annotation or online RL.

Future work includes systematic fine-tuning experiments over successive training rounds, extension to heap-manipulating programs with separation-logic invariants, and integration of symbolic techniques (abstract interpretation, interpolation) for a hybrid symbolic–neural architecture.

References

Dirk Beyer. Competition on software verification (SV-COMP). `https://sv-comp.sosy-lab.org/`, 2023.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 420–432. Springer, 2003.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models write specifications? In *Proc. ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*, 2024.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. In *Science of Computer Programming*, volume 69, pages 35–45, 2007.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. on Formal Methods Europe (FME)*, pages 500–517. Springer, 2001.

Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pages 499–512, 2016.

Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen Carbonell, et al. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.

Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. Grammar-based whitebox fuzzing. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 206–215, 2008.

Jesse Michael Han, Floris Rabe, Yuhuai Wang, Christian Szegedy, and Christian Szegedy. Proof artifact co-training. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengding Huang, Hamish Ivison, Faeze Brahman, Lester James Validad Miranda, Valentina Pyatkin, et al. TÜLU 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Aurelien Rodriguez, Amaury Hayat, Thibaut Lavril, Gabriel Ebner, and Xavier Martinet. HyperTree proof search for neural theorem proving. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. WizardMath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv preprint arXiv:2308.09583*, 2023.

Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of GPT-4. *arXiv preprint arXiv:2306.02707*, 2023.

ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pages 1076–1086, 2012.

Aditya V. Nori, Sherjil Ozair, Sriram K. Rajamani, and Deepak Viber. Efficient synthesis of probabilistic programs. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 208–217, 2015.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Kexin Pei, David Biber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2023.

Sorin Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pages 318–329, 2004.

Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pages 7751–7762, 2018.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 151–164. Springer, 2020.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Christian Szegedy, and Christian Szegedy. Autoformalization with large language models. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MAmmoTH: Building math generalist models through hybrid instruction tuning. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

A LOOPFACTORY: Full DSL Description

A.1 DSL Grammar

A LOOPFACTORY program consists of an initialisation block followed by one or more loops:

$$\textbf{Prog} \ ::= \ \textbf{Init} \ ; \ \textbf{Loop}^{+} \qquad \textbf{Loop} \ ::= \ \texttt{while} \ (g) \ \{ \ \textbf{Body} \ \}$$

$$\textbf{Body} \ ::= \ \big(\ v := e \ \mid \ \texttt{if} \ (g) \ \{S\} \ [\texttt{else} \ \{S\}] \ \mid \ \textbf{Loop} \ \big)^{*}$$

Variables $\mathcal{V}$ are partitioned into immutable *parameters* $\mathcal{P}$ and writable *state variables* $\mathcal{L}$. Body expressions have the form v `op` x with `op` $\in \{+, -, \times, \mathrm{mod}\}$ (/ additionally permitted for nonlinear programs); the guard g is a linear comparison or the constant `1` (for `while(1)` with `break`). **Init** assigns every variable in $\mathcal{L}$ exactly once from expressions over $\mathcal{P}$.

A.2 Multi-Loop and Nesting

The grammar supports *sequential* multi-loop programs (**Loop**$^{+}$), where later loops share state computed by earlier ones, and *nested* loops (a **Loop** inside **Body**), bounded by a maximum depth $D_{\max}$. Each loop is generated independently with its own counter and guard, drawing state variables from the shared pool within budget m.

A.3 Complexity-Controlling Fuel Parameters

Structural complexity is governed by fuel parameters (Table 3), each sampled uniformly from a configurable $[min, max]$ range.

Table 3: Fuel parameters controlling per-program complexity.

Parameter	Controls	Default range
m	Total state variables	≤ 6
$[p_{\min}, p_{\max}]$	Immutable parameters	$[1, 2]$
$[n_{\min}, n_{\max}]$	Sequential loops	$[1, 1]$
$[k_{\min}, k_{\max}]$	Assignments per loop body	$[1, 6]$
$[f_{\min}, f_{\max}]$	Conditional blocks per loop	$[0, 3]$
$D_{\max}$	Maximum nesting depth	1

A.4 Semantic Injection

Purely random loop bodies rarely exhibit the algebraic structure found in real verification benchmarks. LOOPFACTORY addresses this through *semantic injection*: with probability π_{core} (default 0.88), the generator seeds each loop with a *semantic core*—a template that fixes a counter–guard pair (guaranteeing termination), one or more core variable updates maintaining a known invariant, and matching initialisations. Core variables are *write-protected* during the subsequent extension phase, which fills remaining budgets with freely sampled expressions over non-core variables, ensuring the injected invariant is preserved regardless of surrounding code. The remaining $1-\pi_{\text{core}}$ of loops are assembled freely.

The cores span eight semantic families (Table 4).

Table 4: Semantic families of core templates.

Family	**Representative invariant**
Linear conservation	$a + b = N$
Monotone accumulation	$s = \sum_{j=0}^{k-1} f(j)$
Proportional stride	$j = c \cdot i$
Modular counting	$c \equiv t \pmod{T}$
State-machine transition	$state \in \{s_1, \ldots, s_k\}$
Division / GCD	$a = q \cdot d + r$
Multiplicative redistribution	$z + xy = a_0 b_0$
Polynomial recurrence	$zw \geq p^2$

A.5 Corpus

Using LOOPFACTORY we synthesise 2,000 programs spanning all eight families: 1,900 for training and 100 held out as a synthetic test set.

B Detailed Per-Loop Algorithm

B.1 Data Structures

For each loop index i the pipeline maintains: $\mathbf{record}_i$ (loop guard, assignments, initial values, invariable parameters); $\mathcal{T}_i$ (execution trace set); C_i (live candidate pool); C_i^* (certified output).

B.2 Houdini Pruning

Input: Candidate set C, verifier $\mathcal{V}$
Output: Maximal inductive subset $C^* \subseteq C$

$C^* \leftarrow C$
repeat
  $R \leftarrow \{c \in C^* \mid \mathcal{V}(c) = \mathit{fail}\}$
  $C^* \leftarrow C^* \setminus R$
until $R = \emptyset$ **or** $C^* = \emptyset$
return C^*

Algorithm 2: Houdini Pruning (Stage F_4)

B.3 Complete Per-Loop Algorithm

B.4 Role of Static Analysis

Before Phase I, `LoopAnalysis` performs a lightweight static pre-pass to populate $\mathbf{record}_i$: loop guard extraction, unchanged-variable identification (for `\at` invariants), conservative integer bound inference, and record assembly. Frama-C/WP is invoked *only* during Houdini pruning (F_4).

Input: P, record $\mathbf{r}$, traces $\mathcal{T}$, LLM $\mathcal{M}$, verifier $\mathcal{V}$
Output: Annotated code P^* or $\perp$

Phase I — Generation:
$P_{\text{tmpl}} \leftarrow \texttt{InsertTemplate}(P, i)$
$(\sigma, \ell) \leftarrow \texttt{PreparePrompt}(\mathbf{r}, \mathcal{T})$
for $j = 1, \ldots, N$ **in parallel do**
  $(\rho_j, I_j) \leftarrow \mathcal{M}(P_{\text{tmpl}}, \sigma, \ell, t_j)$
$C_{\text{pool}} \leftarrow \bigcup_j \text{conjuncts}(I_j) \ / \sim_{\text{ws}}$

Phase II — Multi-Stage Filtering:
$C_1 \leftarrow \{c \in C_{\text{pool}} \mid \text{SynValid}(c)\}$ ▷ F_1
$C_2 \leftarrow \{c \in C_1 \mid \forall \sigma \in \mathcal{T} : \sigma \models c\}$ ▷ F_2
$C_3 \leftarrow \text{SemDedup}(C_2)$ ▷ F_3
$C_4 \leftarrow \texttt{Houdini}(C_3, \mathcal{V})$ ▷ F_4
$C^* \leftarrow \text{TrivFilter}(C_4)$ ▷ F_5
if $C^* = \emptyset$ **then return** $\perp$

Phase III — Verification & Repair:
if $\mathcal{V}$ confirms postcondition **then**
  return $\texttt{Annotate}(P, C^*)$
else run $\leq K$ repair iterations
return $\texttt{Annotate}(P, C^*)$ or $\perp$

Algorithm 3: InvariantGenerator per loop i

B.5 Non-Attributability of the Certified Output

Definition 2 (Attribution)**.** *C^* is* attributable to call j *if* $C^* \subseteq C_j$.

Proposition 3 (Non-attributability)**.** *If the correct annotation requires $m \geq 2$ conjuncts, each discovered independently with probability q per call, then single-call coverage is q^m while union coverage is $1 - (1-q)^{mN}$. For q=0.6, m=4, N=5: single $\approx$ 0.13; union $\approx$ 1.00.*

Any per-call binary scoring (including GRPO) systematically mislabels calls contributing essential conjuncts to the union.

C DPO Data Construction

C.1 Chosen and Rejected Responses

The chosen annotated program is $P^+ = \texttt{Annotate}(P, i, C^*)$. For each rejected conjunct $r \in C_{\text{pool}} \setminus C^*$: $P_r^- = \texttt{Annotate}(P, i, \{r\})$, labelled by rejection stage $k(r) \in \{1, \ldots, 5\}$. The full DPO record is $\mathcal{D}(P, i, r) = (P,\ P^+,\ P_r^-,\ k(r))$.

C.2 Off-Policy Augmentation

The pipeline applies two complementary off-policy augmentation strategies to enrich the DPO training signal. Both operate purely on the artefacts already produced by the filtering pipeline, requiring no additional LLM or verifier calls.

Negative pool collection. During filtering, every conjunct eliminated at *any* stage is added to a per-program negative pool B:

- **Syntax gate** (F_1): conjuncts violating ACSL syntax rules or containing forbidden operators (`^`, `\pow`).
- **Trace gate** (F_2): conjuncts passing syntax but falsified by at least one execution trace.
- **Houdini gate** (F_4): conjuncts removed during iterative formal pruning (present before Houdini but absent after).

Conjuncts already in the chosen set C^* are excluded from B (by normalised text comparison).

Augmentation algorithm. Algorithm 4 combines both strategies into a unified procedure. Let $m = |C^*|$ and $n = |B|$.

Input: Chosen code P^+ with m conjuncts C^*, negative pool B (n=$|B|$), rejected candidates $\{R_j\}$ from filtering
Output: Set of rejected variants $\{P_i^-\}$

Strategy 1 — Conjunct recombination:
repeat until enough samples collected:
  Sample $g \sim \mathrm{Uniform}(0,\ \max(m{-}2,\ 0))$ ▷ *good count*
  Sample $b \sim \mathrm{Uniform}(0,\ n)$ ▷ *bad count*
  Sample $k_{\max} \sim \mathrm{Uniform}(\max(m{-}2,\ 1),\ m{+}2)$ ▷ *total cap*
  if $g + b < 1$ **or** $g + b > k_{\max}$ **then** reject and resample
  Draw g conjuncts uniformly from $C^* \ \rightarrow G_{\mathrm{sub}}$
  Draw b conjuncts uniformly from $B \ \rightarrow B_{\mathrm{sub}}$
  $P^- \leftarrow \texttt{Annotate}(P,\ G_{\mathrm{sub}} \cup B_{\mathrm{sub}})$
  if $P^- \neq P^+$ **then** emit P^-

Strategy 2 — Error-candidate subset:
for each rejected candidate R_j **do**
  Let $\ell = |\mathrm{conjuncts}(R_j)|$
  Sample $k \sim \mathrm{Uniform}(1,\ \max(\ell{-}2,\ 1))$
  Draw k conjuncts uniformly from $R_j \ \rightarrow S$
  $P^- \leftarrow \texttt{Annotate}(P,\ S)$
  if $\mathrm{conjuncts}(P^-) \neq C^*$ **then** emit P^-

Algorithm 4: Off-policy DPO augmentation

Key properties:

- **Strategy 1** (conjunct recombination): since $g \leq m{-}2$, the good subset alone is always a strict subset of C^*, so the variant cannot equal the chosen annotation even when $b = 0$. The randomised cap $k_{\max}$ keeps the total annotation size close to m, preventing degenerate extremes.
- **Strategy 2** (error-candidate subset): each rejected candidate R_j is a full annotated program that failed at some filtering stage. A random proper subset of its conjuncts produces a new rejected variant, with a normalised check ensuring the conjunct set differs from C^*.
- Both strategies are zero-cost: they recombine existing artefacts without additional model inference or formal verification.

C.3 Stage-Stratified Loss

We extend DPO with stage-dependent weights w_k:

$$\mathcal{L}_{\text{S-DPO}} = -\mathbb{E}\left[w_{k(r)} \cdot \log \sigma\left(\beta \log \frac{\pi_\theta(P^+ \mid P)}{\pi_{\text{ref}}(P^+ \mid P)} - \beta \log \frac{\pi_\theta(P_r^- \mid P)}{\pi_{\text{ref}}(P_r^- \mid P)}\right)\right]$$

with default weights w_1=0.5, w_2=1.0, w_3=1.5, w_4=2.0, w_5=2.5, reflecting that later-stage rejections are harder and more informative contrasts.

D Prompts

D.1 System Prompt

System Prompt

```
You are a formal verification expert specializing in loop
invariant synthesis for Frama-C with the WP plugin.

## TASK
Given a C function with a loop and a postcondition
(/*@ assert ... */), generate ACSL loop annotations
(invariants, assigns) that enable Frama-C/WP to verify
the assertion.

## CORE STRATEGY: Assertion-Driven Invariant Synthesis
1. Decompose the assertion: each conjunct of the assert
   (split on &&) is a candidate loop invariant.
2. Add boundary invariants: weaken the loop guard.
   - while(i < n) needs: i <= n
   - while(n <= a) needs: n <= a+1
3. Preserve parameter values: if a parameter is unmodified,
   assert: loop invariant x == \at(x, Pre);
4. Verify sufficiency: check that (invariants AND NOT guard)
   implies the postcondition.

## FORBIDDEN CONSTRUCTS
- \forall, \exists  (quantifiers not supported)
- predicate, logic, axiomatic, lemma
- Ternary ? :
- \at(local_var, Pre)  (\at only on function parameters)
- ^ for exponentiation  (write x*x*x for x cubed)

## ALLOWED OPERATORS
Comparison: ==, !=, <, <=, >, >=
Logical:    &&, ||, !, ==>, <==>
Arithmetic: +, -, *, /, %

## OUTPUT
Return ONLY the annotated C function. Do not include
explanations.
```

D.2 Loop Context Structure

The user prompt assembles four components into a structured loop context block. Table 5 summarises each field and its source.

The overall prompt layout is shown below:

Table 5: Fields of the loop context prompt and their construction.

Field	Content	Source
Pre-Condition	ACSL state at loop entry	Symbolic execution
Transition Relation	One-step $v \mapsto e'$	Symbolic execution
Loop Snippet	Raw C code of loop body	Static analysis
Execution Traces	Temporally labelled states	Dynamic sampling
Variable List	$V_{\text{mod}} \cup \mathcal{P}$	Static analysis

Loop Context Layout

```
### Loop Context: ###
1. Loop Context
  A. Pre-Condition: <ACSL state expression>
  B. Loop Transition Relation: <one-step summary>
  C. Loop Snippet:
     ‘‘‘c
     <loop code>
     ‘‘‘

2. Execution Traces
  [TRACE 1]
    [state] (BEFORE loop starts) ->
    [state] (BEFORE iteration 1 body executes) ->
    [state] (AFTER loop terminates)
  ...

### AVAILABLE VARIABLES AND PARAMETERS
**Available Variables:** v1, v2, ...
**Function Parameters:**
  - p1 (or \at(p1, Pre) for initial value)
```

D.3 User Prompt

The user prompt contains two parts: a loop context block and execution traces with temporal labels.

User Prompt Template

```
### Loop Context: ###
1. Loop Context
  A. Pre-Condition: ‘<precondition>‘
  B. Loop Transition Relation: ‘<transition_expr>‘
  C. Loop Snippet:
‘‘‘c
<loop_code>
‘‘‘
2. Execution Traces
   WARNING: CRITICAL TEMPORAL SEMANTICS:
   - Each state is labeled as
     ’BEFORE loop body execution’
     or ’AFTER loop terminates’
   - Loop invariants must hold at the START of each
     iteration (BEFORE body executes)
```

Trace Example

```
[TRACE 1]
    [ n==0 && x==0 && y==1 && z==6 ]
        (BEFORE loop starts) ->
```

```
[ n==1 && x==1 && y==7 && z==12 ]
    (BEFORE iteration 1 body executes) ->
[ n==2 && x==8 && y==25 && z==18 ]
    (AFTER loop terminates)
```

Temperature diversity: the primary call uses t_1=1.0; subsequent calls draw uniformly from $\{0.8, 1.0, 1.1, 1.2\}$.

E ACSL Syntax Gate Rules

Each conjunct is checked against these ordered rules (any failure causes rejection):

1. **Trivial/placeholder tokens:** `true`, `false`, `...`, punctuation-only.
2. **Non-ASCII characters:** Unicode inequality signs, control characters.
3. **Nested `loop invariant` keyword:** LLM emits full annotation line instead of bare expression.
4. **Bracket balance:** Unmatched parentheses, brackets, or braces.
5. **Forbidden quantifiers:** `\forall`, `\exists`.
6. **Forbidden ACSL definitions:** `predicate`, `inductive`, `logic`, `axiomatic`, `lemma`.
7. **Forbidden math operators:** `\product`, `\sum`, `\min`, `\max`.
8. **Ternary operator:** `? :` is not valid ACSL.
9. **Undefined variable names:** Checked against known variables from **record**.
10. `\at` **scope:** Only permitted on function parameters $\mathcal{P}$.
11. **Exponentiation:** `^` or `\pow` rejected.

Post-merge sanitisation rewrites local-variable `\at` to plain references and drops any surviving exponentiation tokens.

F Dynamic Sampling Details

F.1 Tiered Value Generation

For each parameter v with domain $[\ell_v, u_v]$:

$$\begin{aligned}
\mathcal{T}_0(v) &= \{0, 1, -1\} \cap [\ell_v, u_v] && \text{(corners)} \\
\mathcal{T}_1(v) &= \{2, \ldots, 10\} \cap [\ell_v, u_v] && \text{(small positives)} \\
\mathcal{T}_2(v) &\subseteq \{11, \ldots, 50\} && \text{(5 medium values)} \\
\mathcal{T}_3(v) &\subseteq \{51, \ldots, 100\} && \text{(3 large values)}
\end{aligned}$$

F.2 Phased Cartesian Sampling

Four phases fill a quota of $S_{\max}$=20 samples: (1) corner-case tuples from $\prod_v \mathcal{T}_0(v)$; (2) mixed tier 0+1 combinations ordered by total tier index; (3) all tiers, prioritised similarly; (4) biased random fill (80% from $[0, 20]$, 20% from $[21, 100]$).

F.3 Trace Formatting

At most $G{=}5$ trace groups per call; within each, at most $k_{\max}{=}10$ iteration snapshots (first and last halves retained if truncated).